

Angular Component and Function Toggle Framework

Design document with JSON-driven component activation, route blocking, function-level controls, Mermaid MMD, and tradeoff analysis.

Core rule: if a component or function is not present in JSON, it is enabled by default.

1. Objective

Build a centralized Angular toggle framework that can enable or disable entire components, prevent navigation to disabled route components, and optionally disable specific functions inside enabled components. The system is controlled by JSON so behavior can change without code cleanup.

2. Problem Statement

Without a framework, each component must manually call ToggleService before running code. This duplicates logic, is easy to forget, and makes toggles hard to audit. We want one reusable infrastructure layer instead of scattered checks.

```
// Avoid repeating this everywhere:
if (!toggleService.isComponentEnabled('UserComponent')) {
  return;
}
```

3. Proposed Architecture

Layer	Responsibility
toggle-config.json	Source of truth; stores only exceptions from default enabled behavior.
ToggleService	Decision engine for component/function status.
Route Guard	Prevents navigation to disabled route components.
@ToggleComponent	Controls/logs component lifecycle logic.
@ToggleFunction	Controls execution of specific component methods.

4. Mermaid MMD - High-Level Architecture

```
graph TD
    A[toggle-config.json] --> B[ToggleService]
    B --> C[Route Guard]
    B --> D[Component Decorator]
    B --> E[Function Decorator]
    C --> F[Block or Allow Route]
    D --> G[Allow or Skip Component Logic]
    E --> H[Allow or Block Function Execution]
    F --> I[User Experience]
    G --> I
    H --> I
```

5. JSON Configuration Design

```
{
  "toggleABC": {
    "enabled": true,
    "components": {
      "UserComponent": {
        "enabled": false,
        "functions": {
          "function1": true,
          "function2": false
        }
      },
      "AdminComponent": {
        "enabled": true
      }
    }
  }
}
```

```

}
}
}

```

Condition	Result
Global enabled = false	Everything disabled.
Component missing from JSON	Component enabled by default.
Component enabled = true	Component enabled.
Component enabled = false	Component disabled.
Function missing from JSON	Function enabled by default.
Function value = false	Function blocked.
Component disabled	All functions inside that component are blocked.

6. Mermaid MMD - Route-Level Runtime Flow

```

sequenceDiagram
    actor User
    participant Router
    participant Guard as Toggle Guard
    participant Service as ToggleService
    participant Component
    User->>Router: Navigate to /user
    Router->>Guard: canActivate()
    Guard->>Service: isComponentEnabled("UserComponent")
    alt Component enabled
        Service-->>Guard: true
        Guard-->>Router: allow
        Router->>Component: create component
    else Component disabled
        Service-->>Guard: false
        Guard-->>Router: redirect /not-allowed
    end

```

7. Mermaid MMD - Function-Level Runtime Flow

```

sequenceDiagram
    actor User
    participant Component
    participant Decorator as Function Decorator
    participant Service as ToggleService
    participant Method as Original Function
    User->>Component: Click button
    Component->>Decorator: function1()
    Decorator->>Service: isFunctionEnabled("UserComponent", "function1")
    alt Function enabled
        Service-->>Decorator: true
        Decorator->>Method: execute original function
    else Function disabled
        Service-->>Decorator: false
        Decorator-->>Component: block execution
    end

```

8. Mermaid MMD - Component Lifecycle Flow

```

sequenceDiagram
    participant Angular
    participant Component
    participant Decorator as Component Decorator
    participant Service as ToggleService
    Angular->>Component: create instance
    Angular->>Decorator: ngOnInit()
    Decorator->>Service: isComponentEnabled(componentName)
    alt Enabled
        Service-->>Decorator: true
        Decorator->>Component: run original ngOnInit()
    else Disabled
        Service-->>Decorator: false
    end

```

```

Decorator-->>Angular: skip component logic
end

```

9. Complete TypeScript Design

ToggleService

```

import { Injectable } from '@angular/core';

export interface ComponentToggle {
  enabled?: boolean;
  functions?: Record<string, boolean>;
}

export interface ToggleConfig {
  toggleABC: {
    enabled: boolean;
    components: Record<string, ComponentToggle>;
  };
}

@Injectable({ providedIn: 'root' })
export class ToggleService {
  private config: ToggleConfig = {
    toggleABC: {
      enabled: true,
      components: {
        UserComponent: {
          enabled: false,
          functions: {
            function1: true,
            function2: false
          }
        },
        AdminComponent: {
          enabled: true
        }
      }
    }
  };

  isComponentEnabled(componentName: string): boolean {
    const root = this.config.toggleABC;
    if (!root.enabled) return false;

    const component = root.components[componentName];
    if (!component) return true; // default enabled

    return component.enabled !== false;
  }

  isFunctionEnabled(componentName: string, functionName: string): boolean {
    const root = this.config.toggleABC;
    if (!root.enabled) return false;

    const component = root.components[componentName];
    if (!component) return true; // component missing = enabled
    if (component.enabled === false) return false;

    const fnValue = component.functions?.[functionName];
    return fnValue !== false; // function missing = enabled
  }
}

```

Route Guard

```

import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { ToggleService } from './toggle.service';

export const toggleGuard: CanActivateFn = (route) => {
  const toggleService = inject(ToggleService);
  const router = inject(Router);
  const componentName = route.data?.['componentName'];

  if (!componentName || toggleService.isComponentEnabled(componentName)) {
    return true;
  }
}

```

```

    }

    console.warn('[ROUTE BLOCKED]', componentName);
    return router.createUrlTree(['/not-allowed']);
  };
};

```

Component Decorator

```

import { ToggleService } from './toggle.service';

export function ToggleComponent(componentName: string) {
  return function (constructor: any) {
    const originalNgOnInit = constructor.prototype.ngOnInit;

    constructor.prototype.ngOnInit = function (...args: any[]) {
      const toggleService: ToggleService = this.toggleService;

      if (!toggleService) {
        console.warn('[ToggleService missing]', componentName);
        return originalNgOnInit?.apply(this, args);
      }

      if (!toggleService.isComponentEnabled(componentName)) {
        console.warn('[COMPONENT DISABLED]', componentName);
        return;
      }

      console.log('[COMPONENT ACTIVE]', componentName);
      return originalNgOnInit?.apply(this, args);
    };
  };
}

```

Function Decorator

```

import { ToggleService } from './toggle.service';

export function ToggleFunction(componentName: string, functionName: string) {
  return function (target: any, key: string, descriptor: PropertyDescriptor) {
    const originalMethod = descriptor.value;

    descriptor.value = function (...args: any[]) {
      const toggleService: ToggleService = this.toggleService;

      if (!toggleService) {
        console.warn('[ToggleService missing]', `${componentName}.${functionName}`);
        return originalMethod.apply(this, args);
      }

      if (!toggleService.isFunctionEnabled(componentName, functionName)) {
        console.warn('[FUNCTION BLOCKED]', `${componentName}.${functionName}`);
        return;
      }

      console.log('[FUNCTION ACTIVE]', `${componentName}.${functionName}`);
      return originalMethod.apply(this, args);
    };

    return descriptor;
  };
}

```

Routes

```

export const routes: Routes = [
  {
    path: 'user',
    component: UserComponent,
    canActivate: [toggleGuard],
    data: { componentName: 'UserComponent' }
  },
  {
    path: 'admin',
    component: AdminComponent,
    canActivate: [toggleGuard],
    data: { componentName: 'AdminComponent' }
  },
  {

```

```

    path: 'not-allowed',
    component: NotAllowedComponent
  }
];

```

Component Usage

```

@ToggleComponent('UserComponent')
@Component({
  selector: 'app-user',
  standalone: true,
  template: `
    <h2>User Component</h2>
    <button (click)="function1()">Function 1</button>
    <button (click)="function2()">Function 2</button>
  `
})
export class UserComponent {
  constructor(public toggleService: ToggleService) {}

  ngOnInit(): void {
    console.log('UserComponent ngOnInit running');
  }

  @ToggleFunction('UserComponent', 'function1')
  function1(): void {
    console.log('function1 executed');
  }

  @ToggleFunction('UserComponent', 'function2')
  function2(): void {
    console.log('function2 executed');
  }
}

```

10. Advantages vs Disadvantages

Advantages	Why it matters
Centralized behavior	All decisions go through ToggleService, so logic is consistent.
Config-driven activation	Remove or update JSON entries without code cleanup.
Default enabled	Only exceptions must be listed; new components continue to work.
Cleaner components	Business logic does not contain repeated toggle checks.
Route protection	Disabled route components are not visible or navigable.
Function-level precision	Specific actions can be blocked without disabling the full component.
Operational kill switch	Teams can quickly disable broken features by config.

Disadvantages	Mitigation
Decorator required	Add only to components/functions that need toggle control.
Not fully automatic	Angular does not expose a supported global lifecycle interceptor.
String-name drift risk	Use constants or helper functions for component names.
Decorator cannot block rendering alone	Use route guard for routed components and structural directive for nested components.
More framework code	Keep ToggleService small and thoroughly unit tested.
Runtime JSON risk	Validate config schema and add safe defaults.

11. Why We Should Add Toggle to Component Decorator

The component decorator is the cleanest place to attach component-level toggle behavior because it keeps toggle logic out of component business code while still making the component explicitly opt in to the framework.

Manual Checks in Component	Decorator-Based Toggle
Scattered in every component	Central reusable behavior
Easy to forget checks	Explicit annotation shows component is controlled
Hard to audit	Search for @ToggleComponent
Cleanup requires code edits	Remove JSON entry to re-enable
Inconsistent implementation possible	One implementation in decorator/service

12. Recommended Decision

Adopt the toggle framework with JSON as the source of truth, ToggleService as the decision engine, route guard for route blocking, @ToggleComponent for component lifecycle control, and @ToggleFunction for method-level control.

13. Final Design Principle

Code = stable infrastructure
JSON = dynamic activation control
Missing from JSON = enabled by default